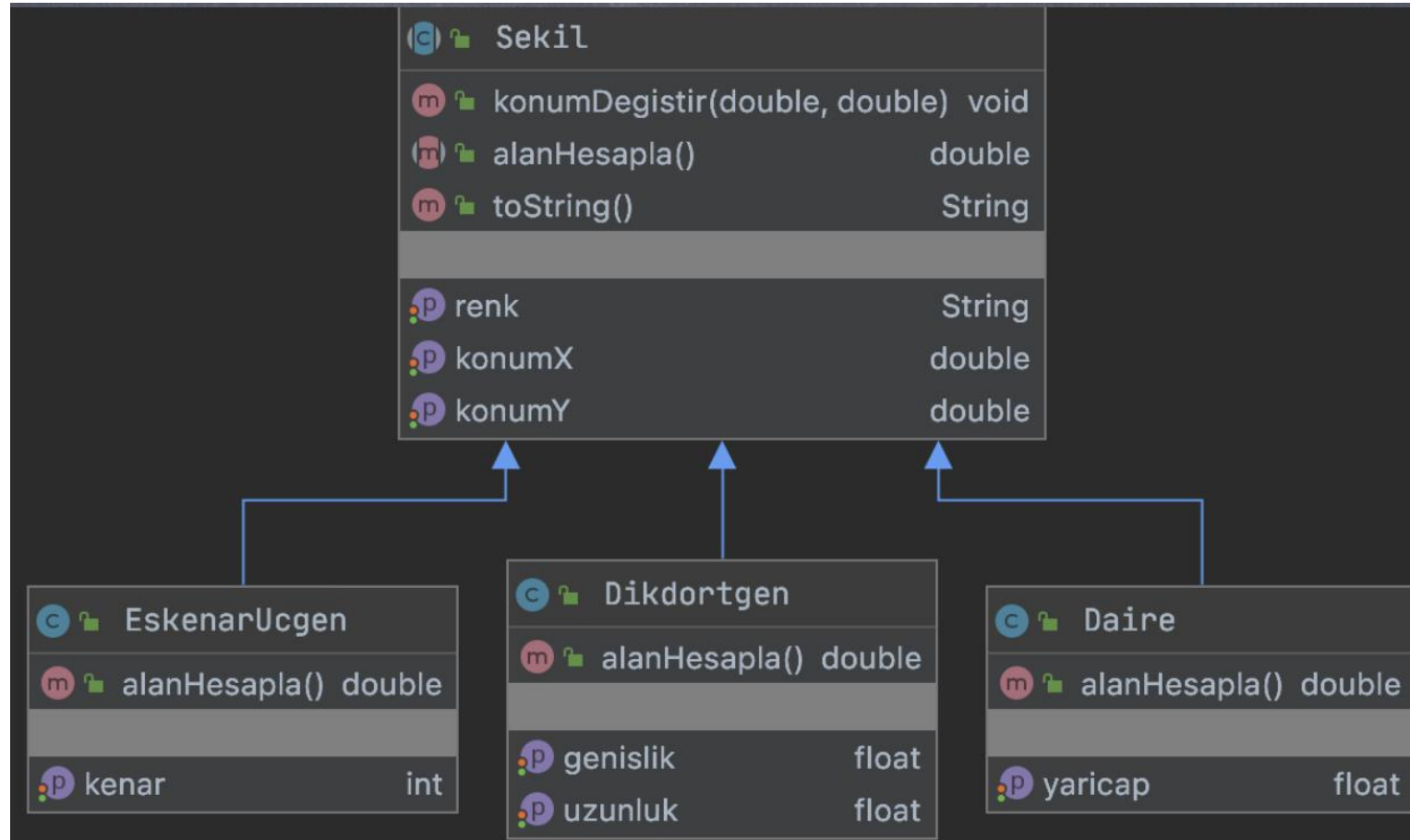


NDAT

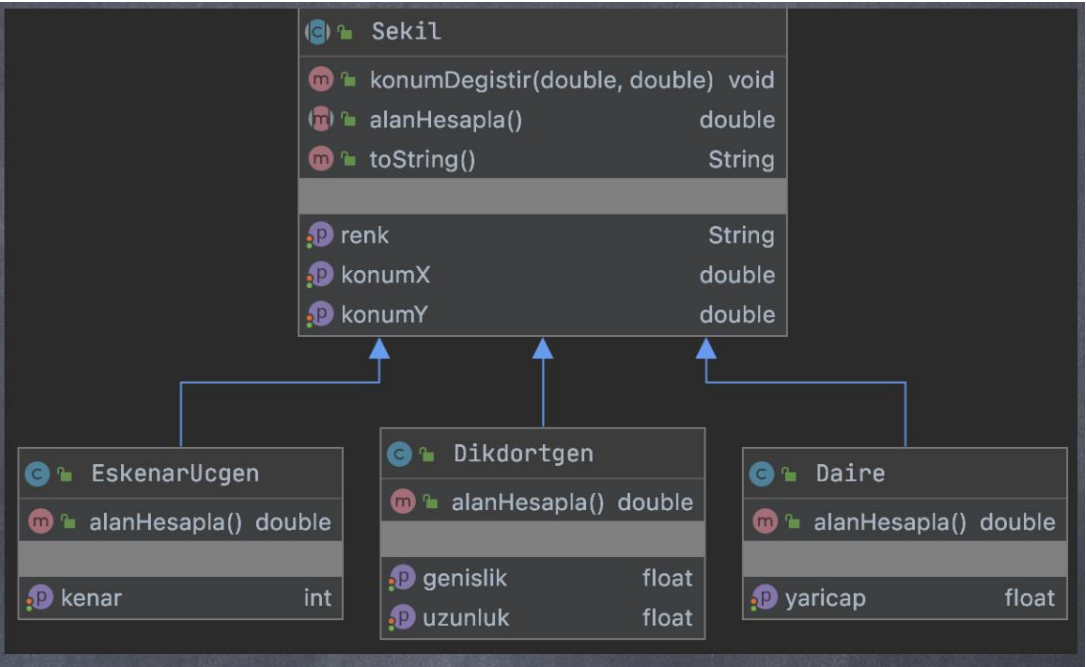
5. hafta

Kalıtım (UML)



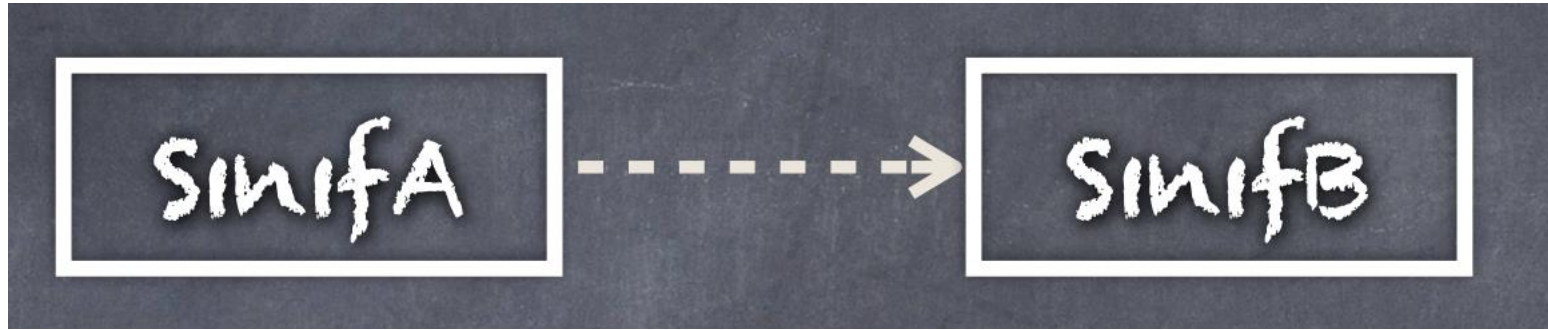
Kalıtım Kod Dönüşümü

```
abstract public class Sekil { //Soyut yöntem sadece soyut sınıflar  
    private double konumX;  
    private double konumY;  
    private String renk;  
  
    public Sekil(double konumX, double konumY, String renk) {  
        this.konumX = konumX;  
        this.konumY = konumY;  
        this.renk = renk;  
    }  
  
    public double getKonumX() { return konumX; }  
  
    public void setKonumX(double konumX) { this.konumX = konumX; }
```

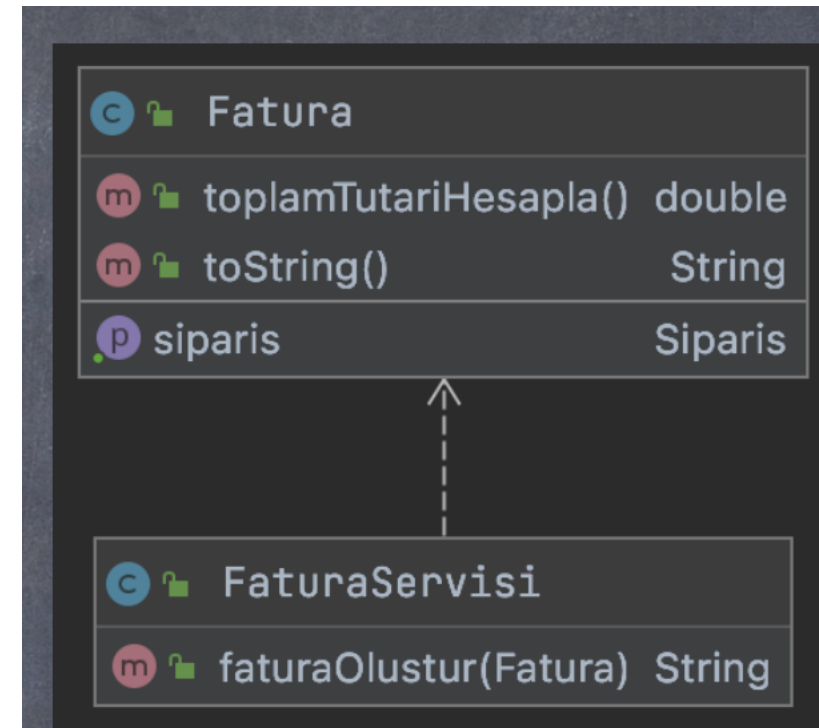


```
public class Daire extends Sekil {  
  
    private float yaricap;  
  
    public Daire(double konumX, double konumY, String renk, float yaricap) {  
        super(konumX, konumY, renk);  
        this.yaricap = yaricap;  
    }  
  
    public float getYaricap() { return yaricap; }  
  
    public void setYaricap(float yaricap) { this.yaricap = yaricap; }
```

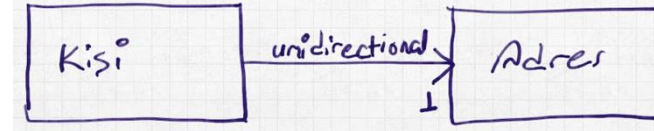
Dependency Bağlantısı



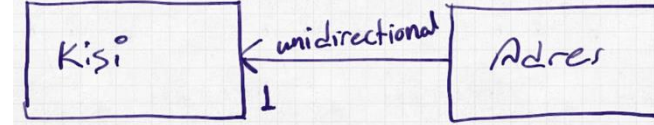
```
public class FaturaServisi {  
  
    public String faturaOlustur(Fatura fatura){  
  
        String faturaXML="<XML>"+fatura.toString();  
        return faturaXML;  
    }  
}
```



Association Bağlantısı



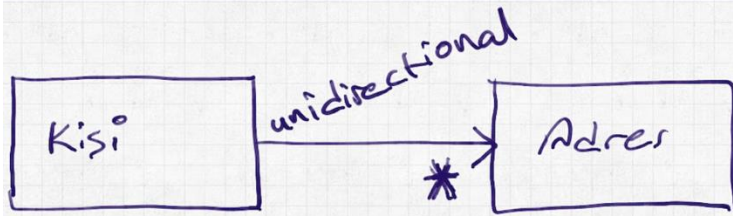
```
class Kişi
{
    private Adres adres;
    ...
}
```



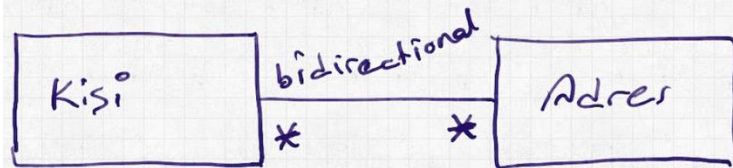
```
class Adres
{
    private Kişi kişi;
    ...
}
```



```
class Kişi
{
    private Adres adres;
    ...
}
class Adres
{
    private Kişi kişi;
    ...
}
```

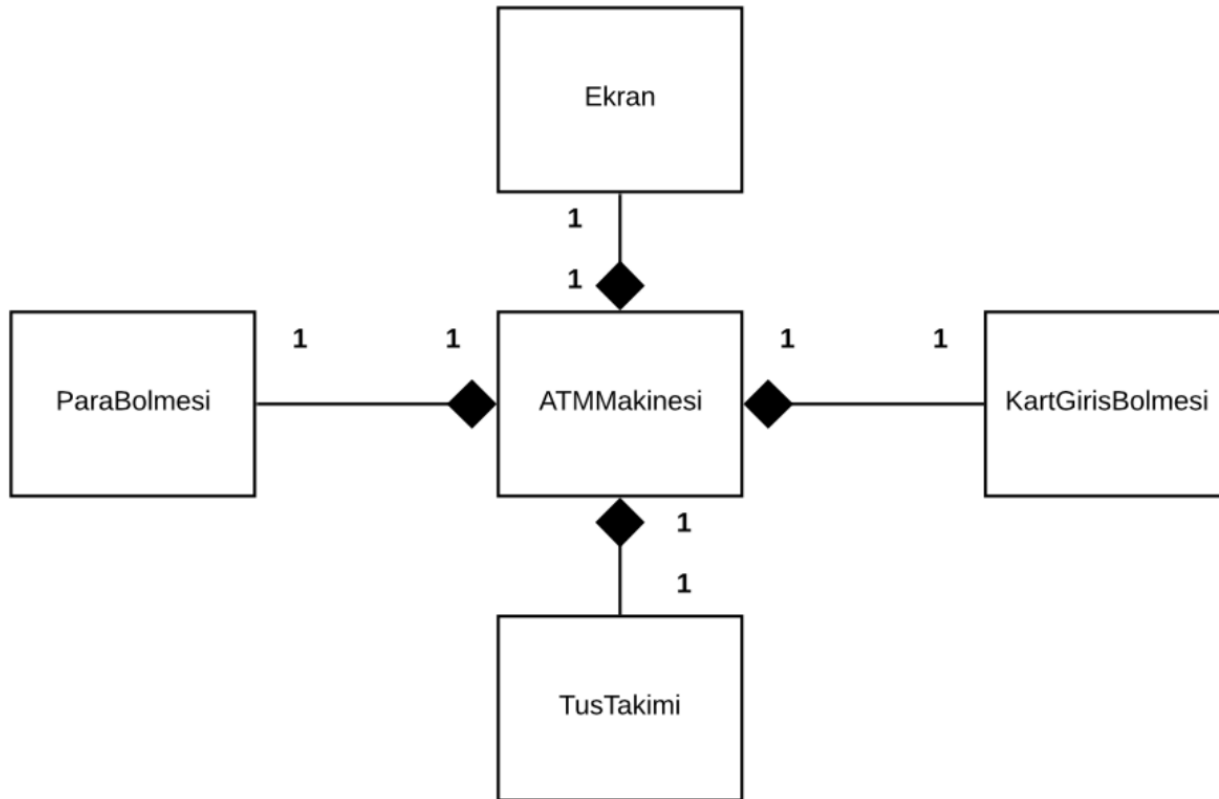


```
class Kişi
{
    private List<Adres> adresler;
    ...
}
```



```
class Kişi
{
    private List<Adres> adresler;
    ...
}
class Adres
{
    private List<Kişi> kisiler;
    ...
}
```

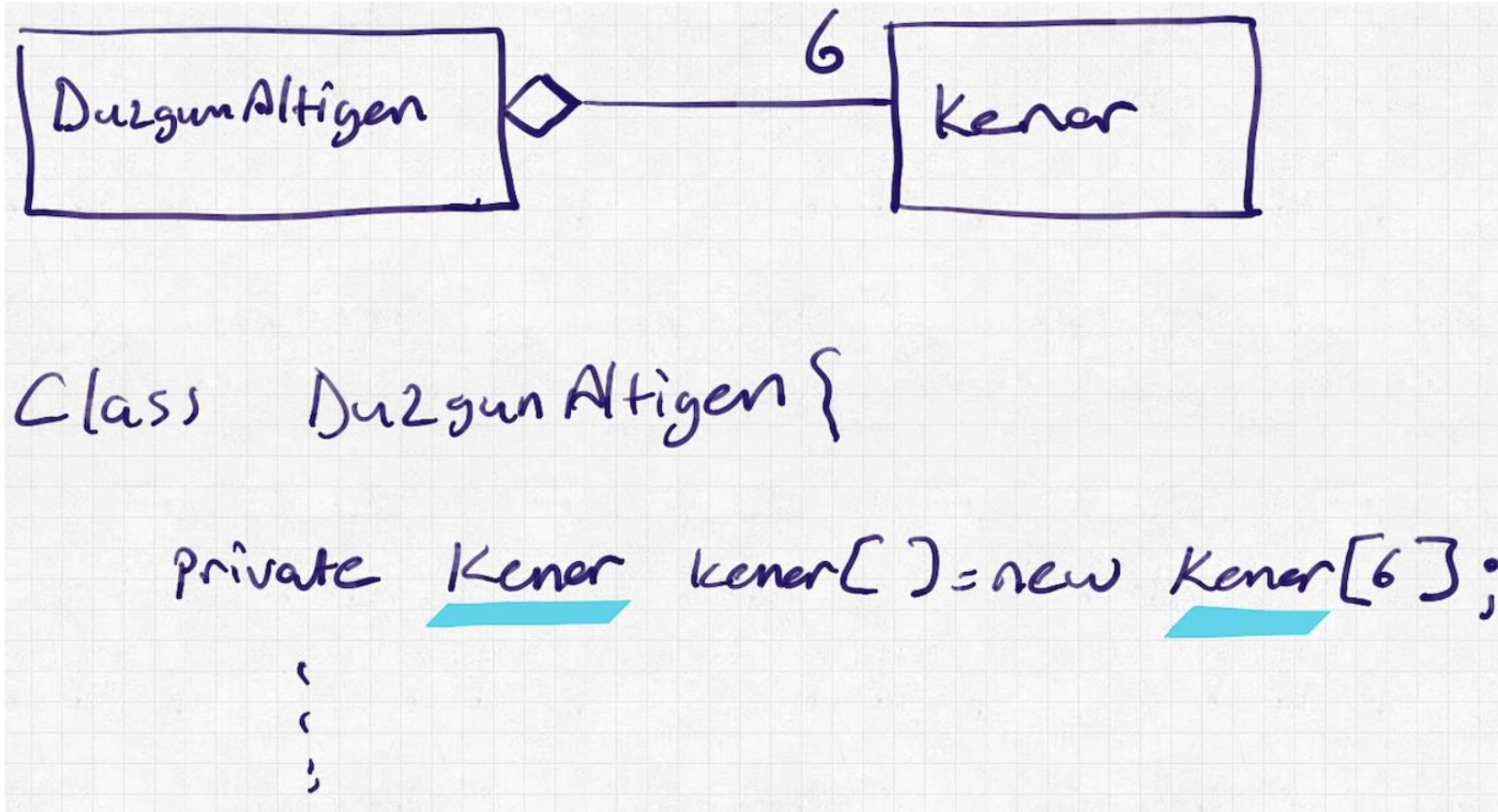
Composition Bağıntısı



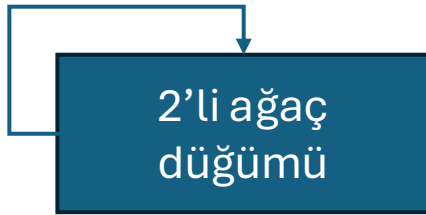
```
public class ATMMakinesi
{
    private Ekran ekran;
    private TusTakimi tusTakimi;
    private ParaBolmesi paraBolmesi;
    private KartGirisBolmesi kartBolmesi;

    public ATM() {
        ekran=new Ekran();
        tusTakimi=new TusTakimi();
        paraBolmesi=new ParaBolmesi();
        kartBolmesi=new KartGirisBolmesi();
    }
}
```

Aggregation Bağıntısı



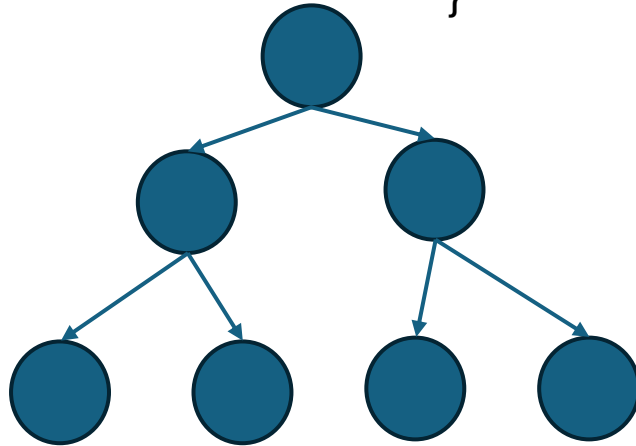
Reflexive Bağıntısı



Class İkiliAgacDugumu{

private İkiliAgacDugumu sol;
private İkiliAgacDugumu sag;

}

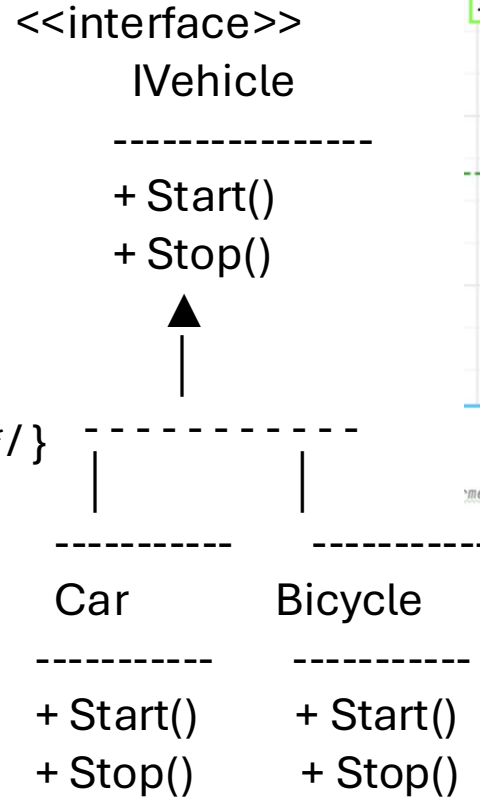


Interface

```
interface IVehicle {  
    void Start();  
    void Stop();  
}
```

```
class Car : IVehicle {  
    public void Start() { /* Motoru çalıştır */ }  
    public void Stop() { /* Fren yap */ }  
}
```

```
class Bicycle : IVehicle {  
    public void Start() { /* Pedallamaya başla */ }  
    public void Stop() { /* Kontrayı sık */ }  
}
```



```
interface IVeritabaniSurucu {  
  
    public void baglan();  
    public void sorguCalistir();  
    public void baglantiSonlandir();  
}
```

I VeritabaniSurucu

+ baglan() void
+ sorguCalistir() void
+ baglantiSonlandir() void

MySQLSurucu

+ baglantiSonlandir() void
+ sorguCalistir() void
+ baglan() void

PostgreSQLSurucu

+ baglantiSonlandir() void
+ sorguCalistir() void
+ baglan() void

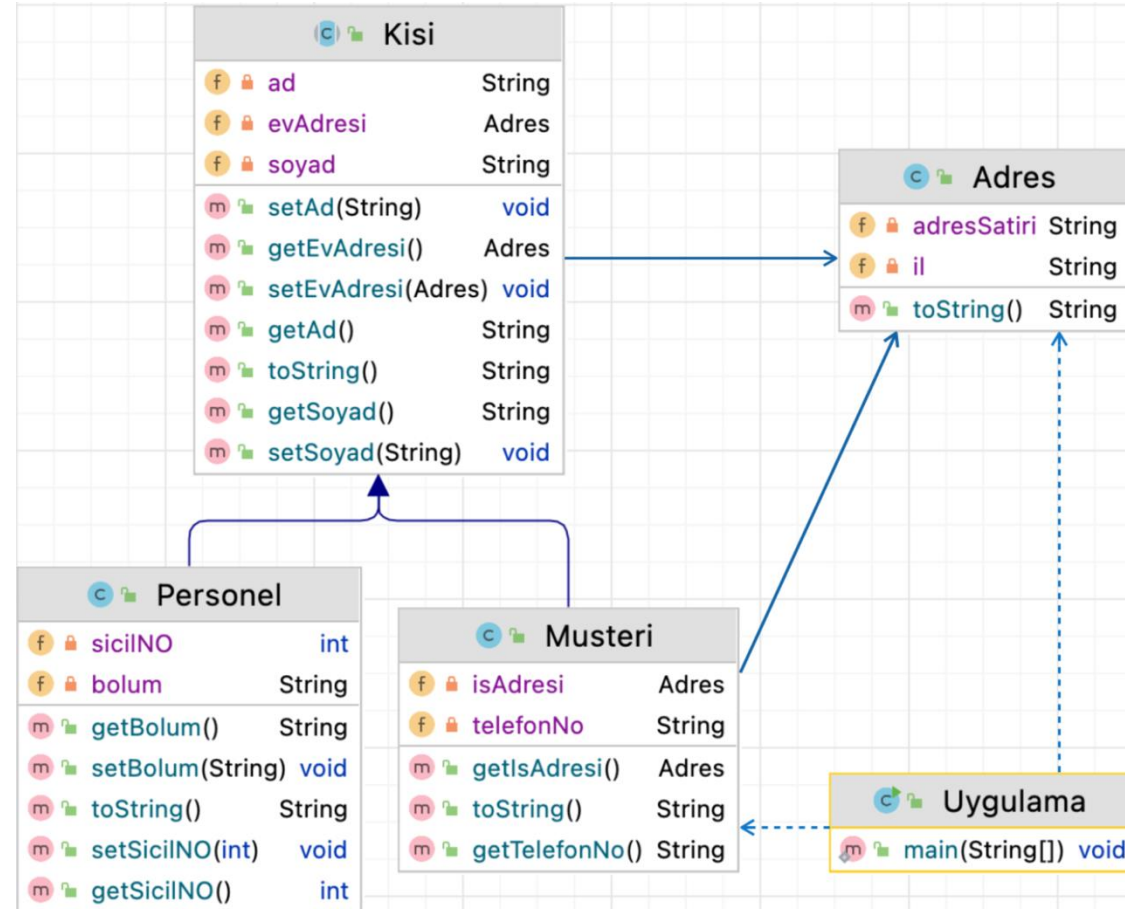
```
public class PostgreSQLSurucu implements IVeritabaniSurucu {  
  
    public void baglan() {  
        System.out.println("PostgreSQL veritabanına bağlanıyor...");  
    }  
  
    public void sorguCalistir() {  
        System.out.println("PostgreSQL sorgusu çalıştırılıyor...");  
    }  
  
    public void baglantiSonlandir() {  
        System.out.println("PostgreSQL veritabanı bağlantısı sonlandırılıyor...");  
    }  
}
```

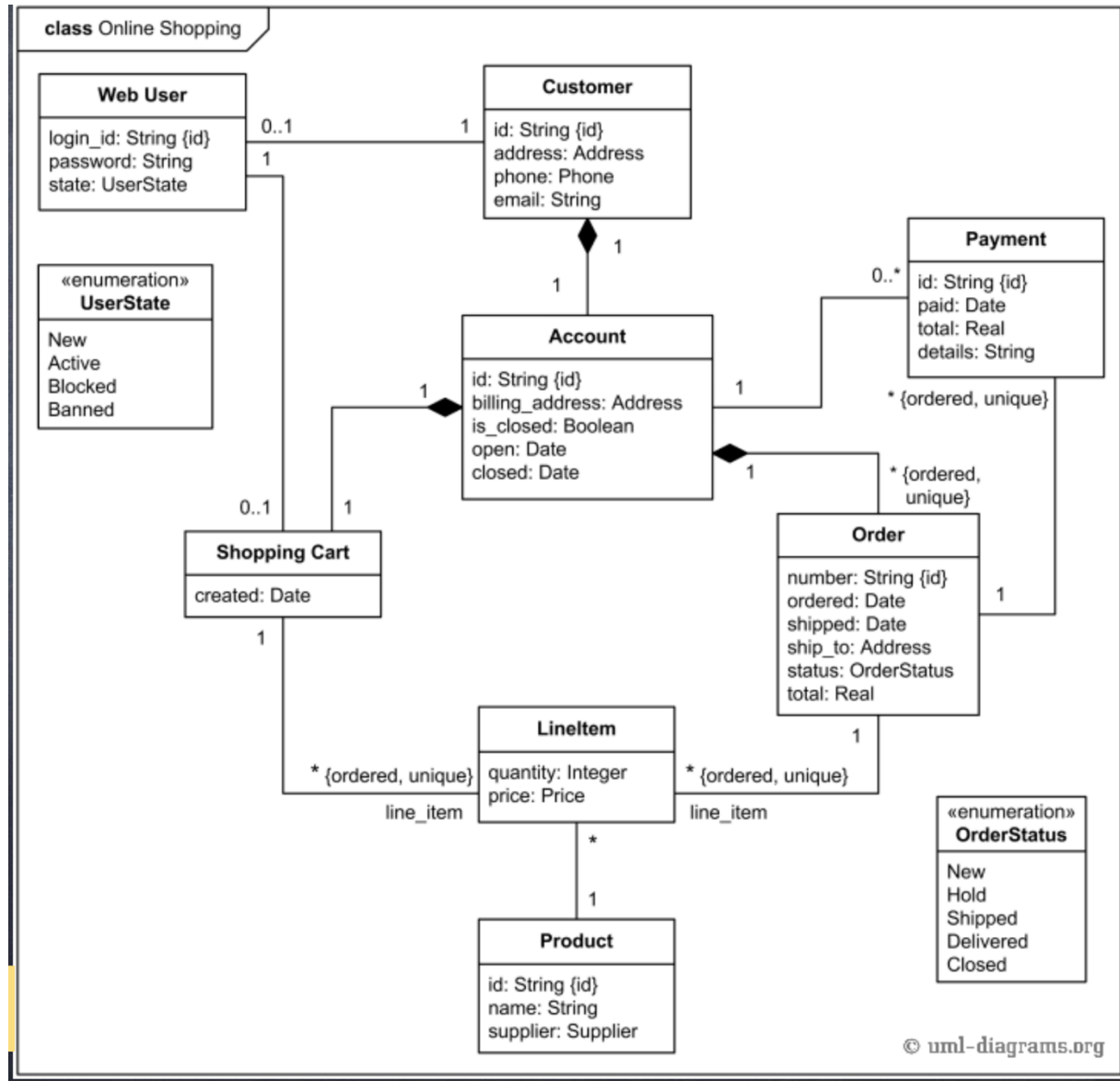
Interface

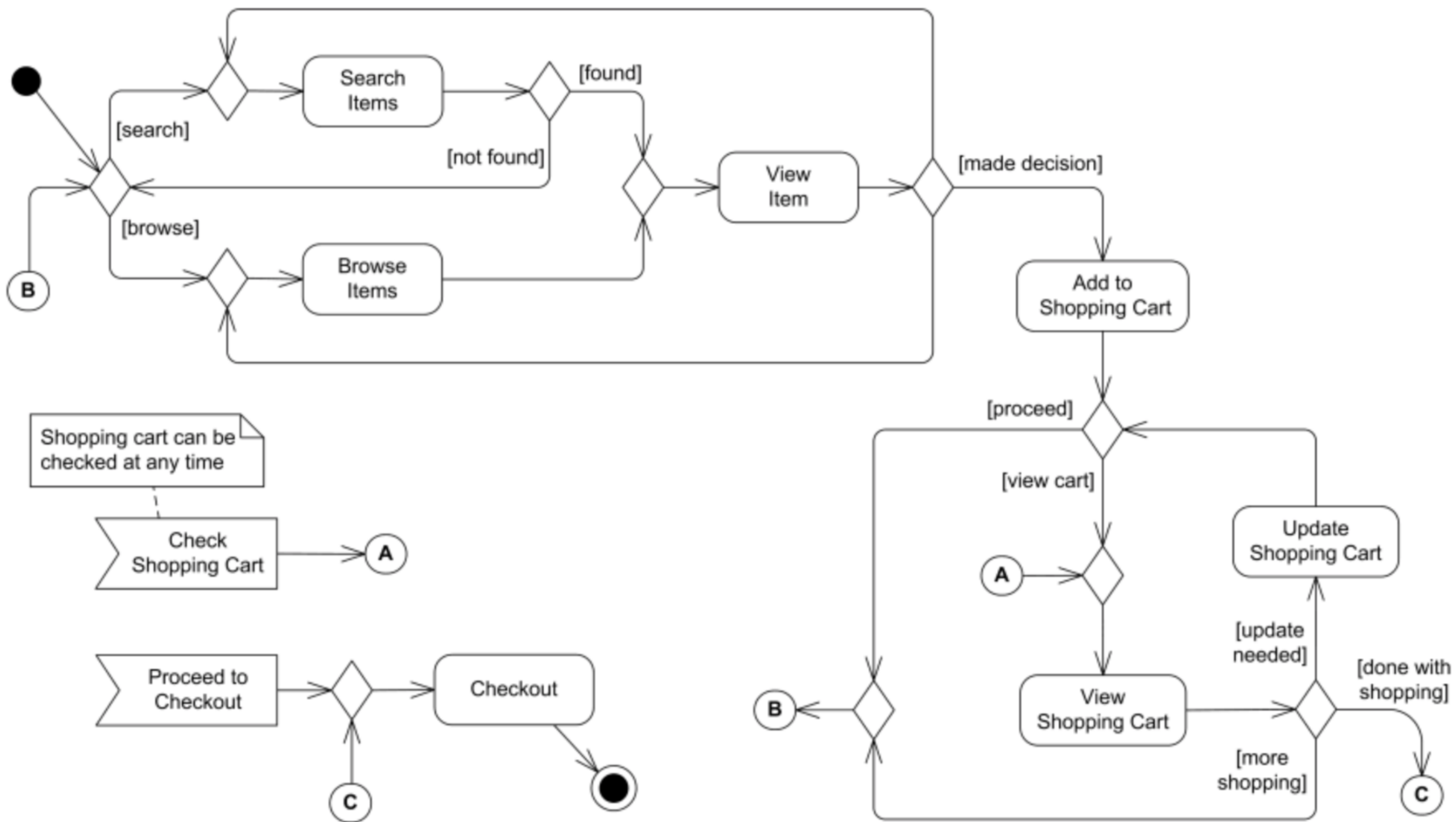
Arayüzler sayesinde:

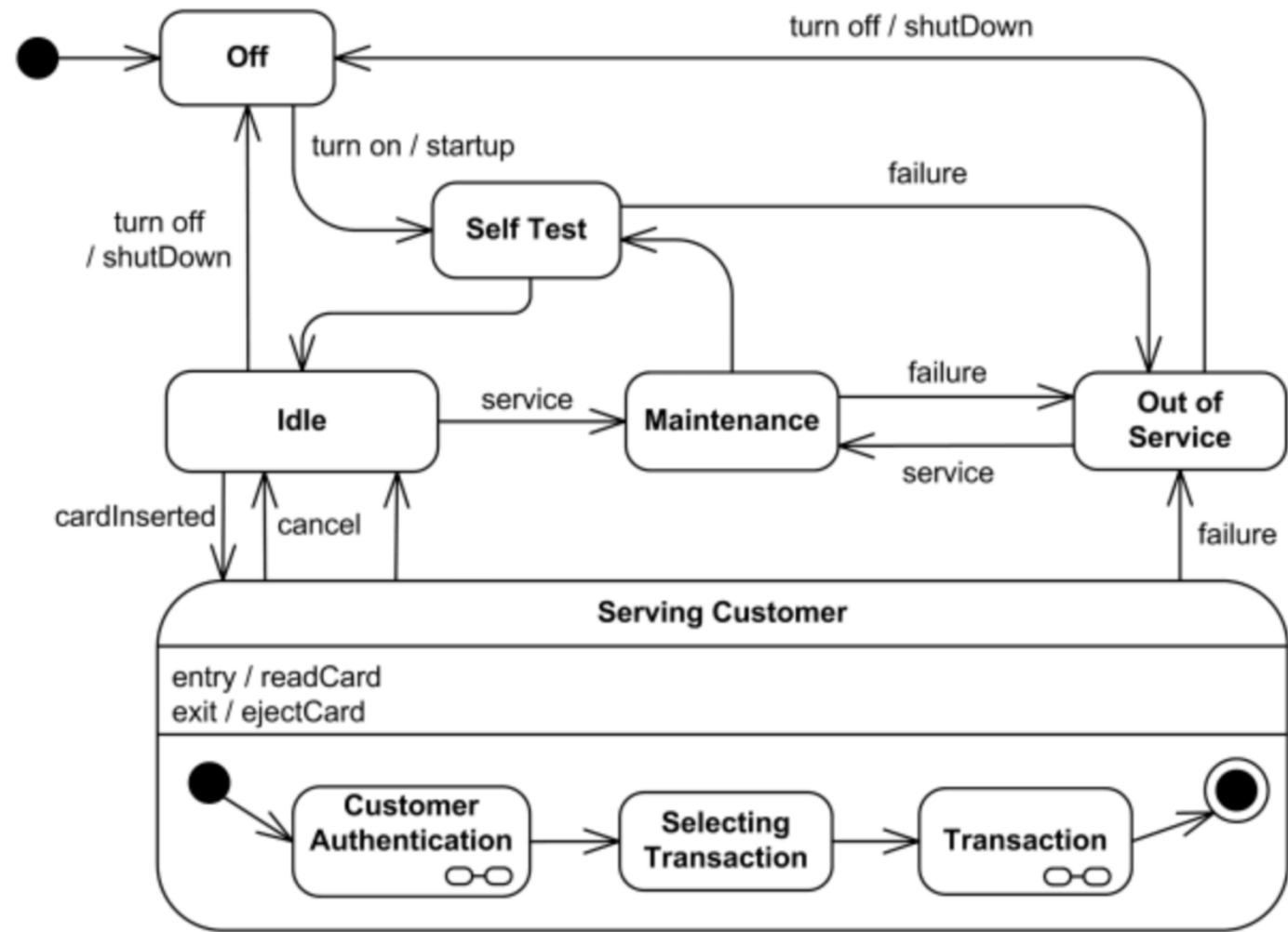
- gerçeklemeler(sınıflar) henüz ortada yok iken istemci kod içerisinde kullanılabilir
- gerçekleştirme ve istemci modüller aynı anda farklı kodlayıcılar tarafından
- oluşturulabilir (takım çalışması)
- yeni özellik eklendiğinde istemci kod
- değiştirilmez (sözleşme yapılıır)
- istemci kod, gerçekleştirme kısmındaki
- değişikliklerden etkilenmez (bağımlılığın
- zayıflatılması)
- aynı istemci kod farklı modüller tarafından defalarca kullanılabilir (kod
- tekrar kullanımı-code reuse)

Örnek









CRC Kartları

- **CRC kartları (Class–Responsibility–Collaborator)**, nesne yönelimli tasarımda sınıfları **çok basit ve görsel bir şekilde düşünmek için kullanılan bir yöntemdir**. Özellikle yazılım tasarımının erken aşamalarında veya derslerde kavramı anlatmak için çok faydalıdır.
- Aslında fikir çok basit:
Her **sınıfı bir kart gibi düşünürüz** ve o kart üzerinde üç şey yazar:
- **Class (Sınıf)**
- **Responsibilities (Sorumluluklar)**
- **Collaborators (İşbirliği yaptığı sınıflar)**

- **Neden CRC Kartları Kullanılır?**
- CRC kartlarının amacı:
- ✓ Sınıfların görevlerini netleştirmek
- ✓ Sınıflar arası ilişkileri görmek
- ✓ Çok büyük tasarımları basitleştirmek
- ✓ UML'e geçmeden önce tasarım yapmak

- **CRC Kartları Nasıl Hazırlanır?**

- Genellikle şu adımlar izlenir:

- | |
|---|
| 1 |
| 2 |
| 3 |
| 4 |

 Sistem senaryosu yazılır
- | |
|---|
| 2 |
|---|

 Senaryodaki **isimler** → **sınıf adayları olur**
- | |
|---|
| 3 |
|---|

 Senaryodaki **fiiller** → **sorumluluklar olur**
- | |
|---|
| 4 |
|---|

 Sınıfların birbirine ihtiyaç duyduğu yerler → **collaborator**

CRC kartları

- -----
- Class: Car
- -----
- Responsibilities:
- - Motoru çalıştırmak
- - Aracı durdurmak
- - Hızlanmayı kontrol etmek
- Collaborators:
- - Engine
- - BrakeSystem
- -----

ATM (Banka müşterilerine finansal servis sağlar)

Sorumluluk

uygulamanın başlatılması

mesaj yazdır (mesajGoruntule())

kullanıcı verisi al (veriAl())

işlem (transaction) yap (islemYap())

kullanıcı Doğrula

kart işlemleri

para işlemleri

ana menü ve işlem seçimi

İşbirliği Yapılan Sınıf

kendisi

Ekran

TusTakimi

Islem (ParaCekme, ParaYatirma,
BakiyeGoruntuleme)

BankaBilgiSistemi, MusteriHesabi

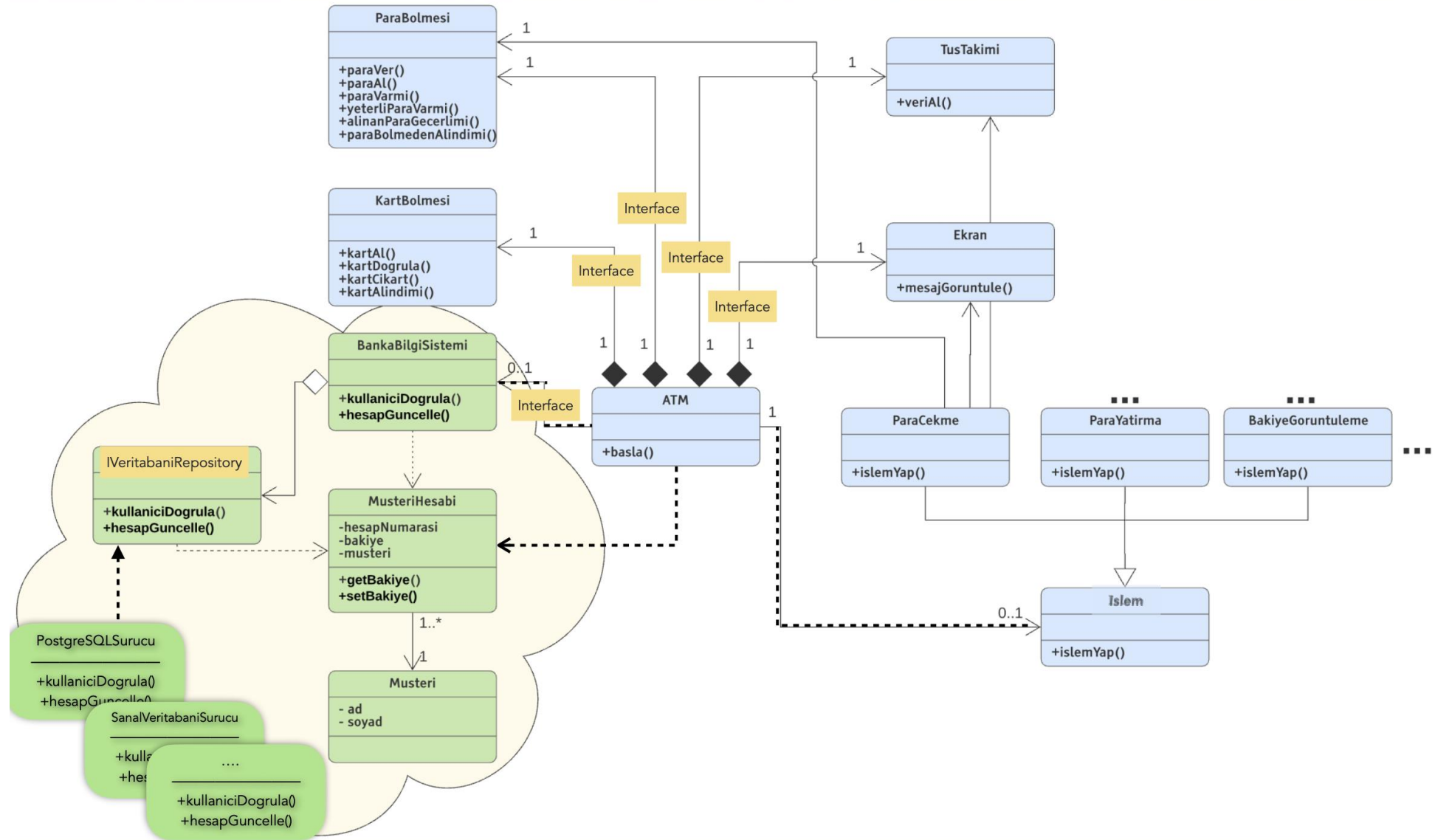
KartBolmesi

ParaBolmesi

kendisi

CRC Kartları

ParaCekme (Para çekme işlemini gerçekleştirir.)	
Sorumluluk	İşbirliği Yapılan Sınıf
para çekme işlemi	kendisi
mesaj yazdır	Ekran
kullanıcı verisi al	TuS Takimi
hesap güncelleme	BankaBilgiSistemi, MusteriHesabi
para işlemleri	ParaBolmesi



UML - Haberleşme(İletişim) Şeması (Communication(collaboration) Diagram)

- **Communication diyagramı**, nesnelerin birbirleriyle **hangi mesajları göndererek iletişim kurduğunu** gösteren bir UML davranış diyagramıdır. Sistem içindeki nesneler arasındaki **etkileşim ve işbirliği** görselleştirilir.
- **Amaçları**
 - Nesneler arasındaki **mesaj alışverişini göstermek**
 - Sistem bileşenleri arasındaki **işbirliğini anlamak**
 - Bir senaryoda **hangi nesnenin hangi işlemi başlattığını** görmek

Özellikleri

- Nesneler **bağlantılar (link)** ile gösterilir.
- Mesajlar **numaralandırılarak** sıralama belirtilir.
Örnek: 1: login(), 2: verifyUser()
- **Sequence diyagramına benzer**, ancak zaman eksenini yerine nesneler arası ilişkiye odaklanır.

Kullanıldığı Yerler

- Nesneler arası **etkileşimi analiz etmek**
- Sistem davranışını **senaryo bazlı incelemek**
- **Nesne yönelimli tasarımda** işbirliğini modellemek

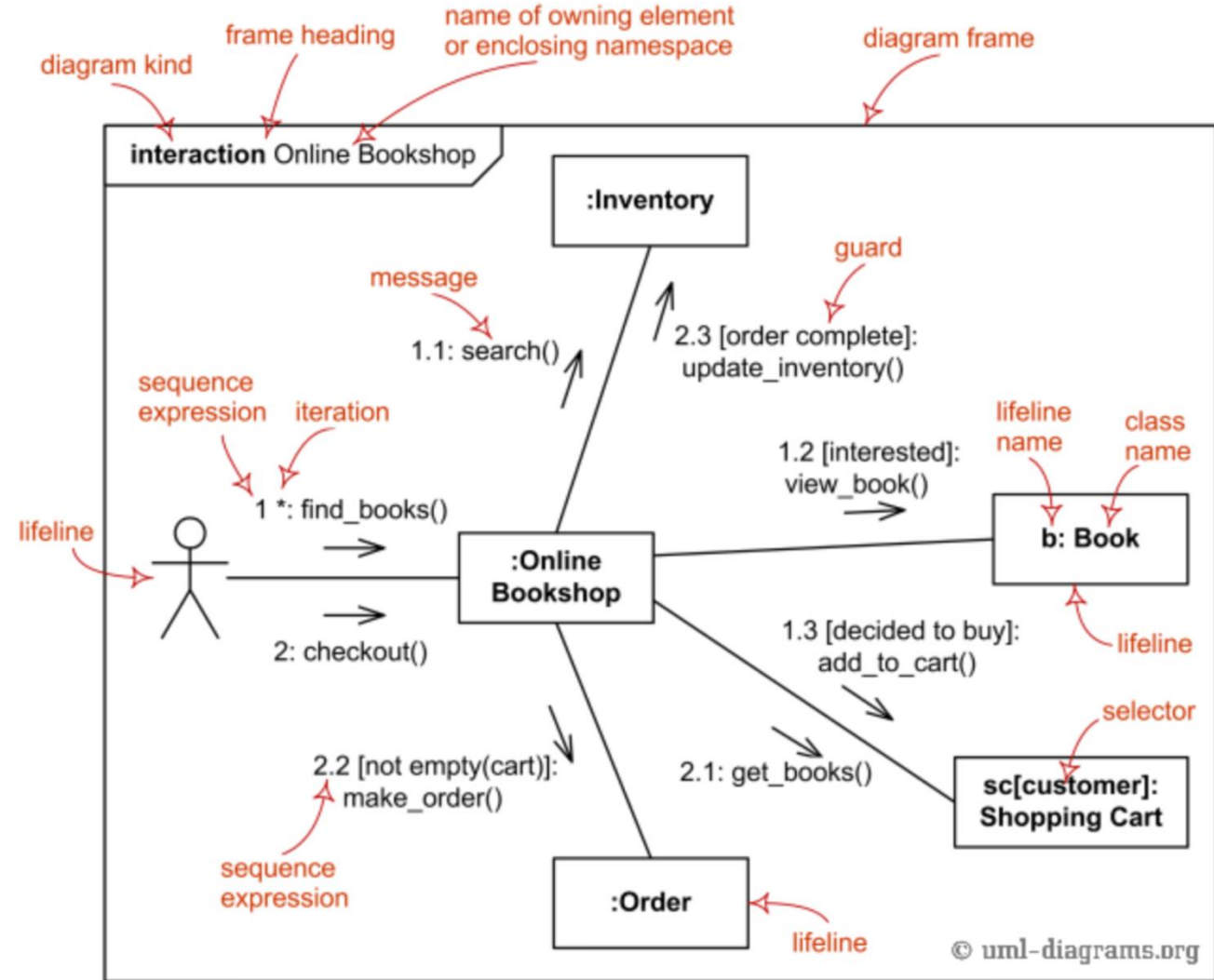
Haberleşme(İletişim) Şeması

Temel Elemanlar

- **Object (Nesne)**
- **Link (Bağlantı)**
- **Message (Mesaj)**

Haberleşme

- Müşteri, alınacak kitabın bulunması için, OnlineBookshop tipli nesnenin find_books() yöntemini çağırır.
- 1.1. OnlineBookshop nesnesi, kitabın aranması için Inventory nesnesinin search() yöntemini çağırır.
- 1.2. OnlineBookshop nesnesi, bulunan kitabın ayrıntıları için b nesnesinin view_book() yöntemini çağırır.
- 1.3. Alınması kararlaştırılan kitabın alışveriş kartına eklenmesi için, OnlineBookshop nesnesi, sc[customer] nesnesinin add_to_cart() yöntemini çağırır.



The major elements of UML communication diagram.

UML - Haberleşme (İletişim) Şeması

Communication (Collaboration) Diagram

Communication diyagramı, nesnelerin birbirleriyle hangi mesajları göndererek iletişim kurduğunu gösteren bir UML davranış diyagramıdır. Sistem içindeki nesneler arasındaki etkileşim ve işbirliği görselleştirilir.

Amaçları

- ◆ Nesneler arasındaki **mesaj alışverişini göstermek**
- ◆ Sistem bileşenleri arasındaki **işbirliğini anlamak**
- ◆ Bir senaryoda hangi nesnenin hangi işlemi başlattığını görmek

Kullanıldığı Yerler

- ◆ Nesneler arası **etkileşimi analiz** etmek
- ◆ Sistem davranışını senaryo bazlı incelemek
- ◆ Nesne yönelimli tasarımda **işbirliğini modellemek**

Özellikleri

- ◆ Nesneler **bağlantılar** (link) ile gösterilir.
- ◆ Mesajlar **numaralandırılarak** sıralama belirtilir.
Örnek: 1: login(), 2: verifyUser)
- ◆ Sequence diyagramına **benzer, ancak zaman eksenini** yerine nesneler arası ilişkiye odaklanır.

Temel Elemanlar

- ◆ **Object** (Nesne)
- ◆ **Link** (Bağlantı)
- ◆ **Message** (Mesaj)

Kaynakça

- Celal Çeken Ders Notları
- Cemil Öz Ders Notları